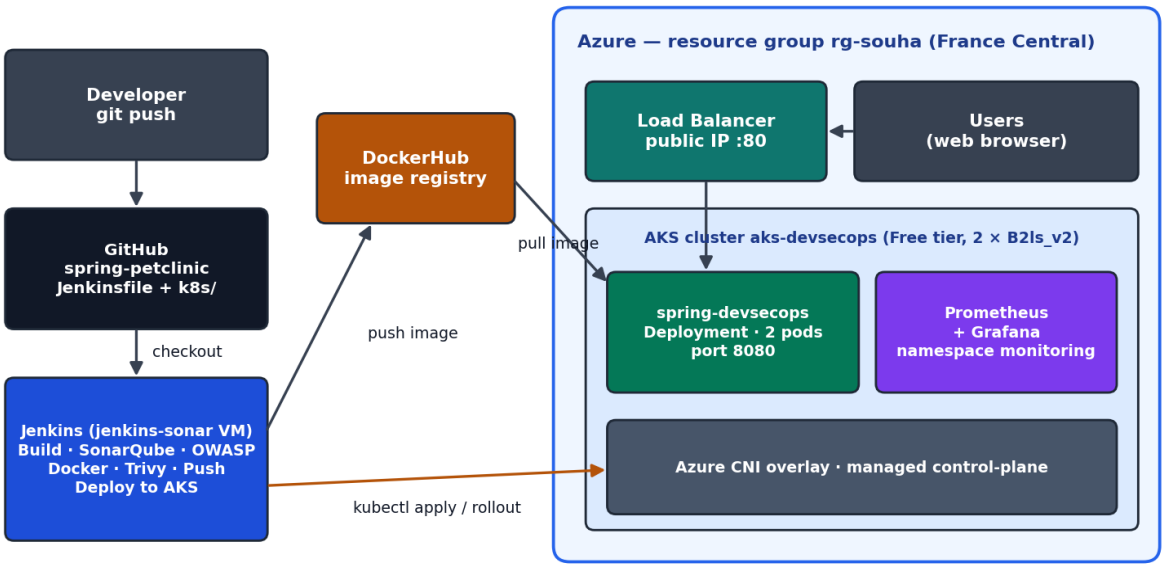
PetShop DevSecOps Pipeline

Project Report — Sprints 0 to 8

Jenkins · SonarQube · OWASP Dependency-Check · Trivy · Docker · Kubernetes · Ansible · Prometheus · Grafana · Microsoft Azure (AKS)



Author: Souhajo | Date: October 2026

1. Project Overview

This project builds a complete DevSecOps chain for the open-source **Spring PetClinic** Java application. Every code change is compiled, analysed for quality and security, packaged in a Docker image, scanned and published to DockerHub. The application was first deployed automatically on a local Kubernetes cluster (kubeadm + Ansible) and monitored with Prometheus and Grafana; in the last sprint it was moved to a managed **Azure Kubernetes Service (AKS)** cluster and the Jenkins pipeline now deploys to Azure by itself. The work was organised in nine agile sprints (0 to 8); each section below lists the objective, what was done, the deliverable and the supporting figures.

Infrastructure

Component	Location	Role
jenkins-sonar	Local VM, 192.168.56.10	Jenkins (port 8090), SonarQube (9000), Docker, Trivy, Ansible
k8s-master / k8s-worker	Local VMs, 192.168.56.11 / .12	Local Kubernetes cluster built with kubeadm (Sprints 5–7)
aks-devsecops	Azure, France Central	Managed Kubernetes cluster, 2 nodes Standard_B2ls_v2 (Sprint 8)

Sprint summary

Sprint	Topic	Deliverable
0	Environment (VirtualBox, Vagrant)	3 VMs running and reachable over SSH
1	Jenkins, Docker, Trivy	CI server ready
2	SonarQube	Jenkins and SonarQube connected
3	Base pipeline	Green pipeline up to the Quality Gate
4	Security scanning	Full CI + security pipeline, image on DockerHub
5	Kubernetes (kubeadm)	2-node local cluster, all nodes Ready
6	Ansible deployment	Application deployed automatically on local K8s
7	Verification & monitoring	App reachable, metrics visible in Grafana
8	Deployment on Azure (AKS)	Public application on AKS, deployed by Jenkins, monitored

Sprint 0 — Environment Preparation

Objective: have all tools installed and three virtual machines ready.

Work done

- Installed VirtualBox and Vagrant and verified their versions.
- Created the project folder *petshop-devsecops* and its *Vagrantfile* describing three Ubuntu 22.04 VMs on a host-only network.
- Started the VMs with `vagrant up` and checked them with `vagrant status`.
- Confirmed SSH access to every VM with `vagrant ssh`.

Deliverable: 3 VMs running, reachable over SSH, with working private IPs (192.168.56.10 / .11 / .12).

Figures

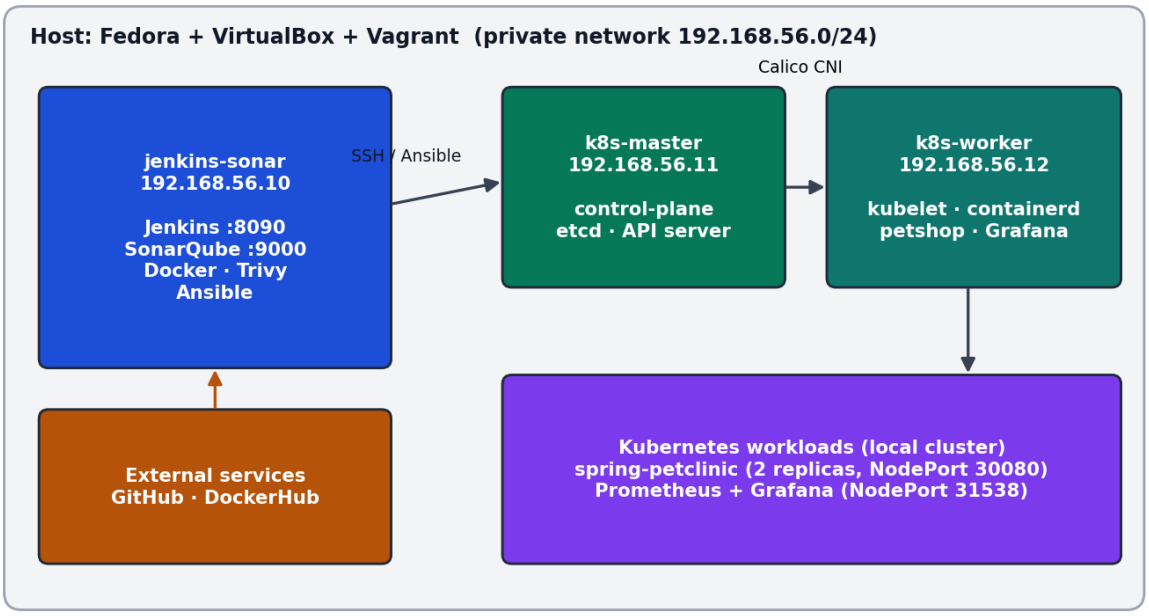


Figure 1 — Architecture of the local lab (three VMs on a private network).

Sprint 1 — Jenkins, Docker and Trivy Setup

Objective: prepare the CI server on the jenkins-sonar VM.

Work done

- Installed JDK 17 and Jenkins; moved Jenkins to port **8090** through a systemd override.
- Installed Docker and gave the *jenkins* user access to the Docker daemon.
- Installed Trivy as vulnerability scanner.
- Unlocked Jenkins in the browser and installed the suggested plugins.

Deliverable: Jenkins accessible and functional; Docker and Trivy available on the command line.

Figures

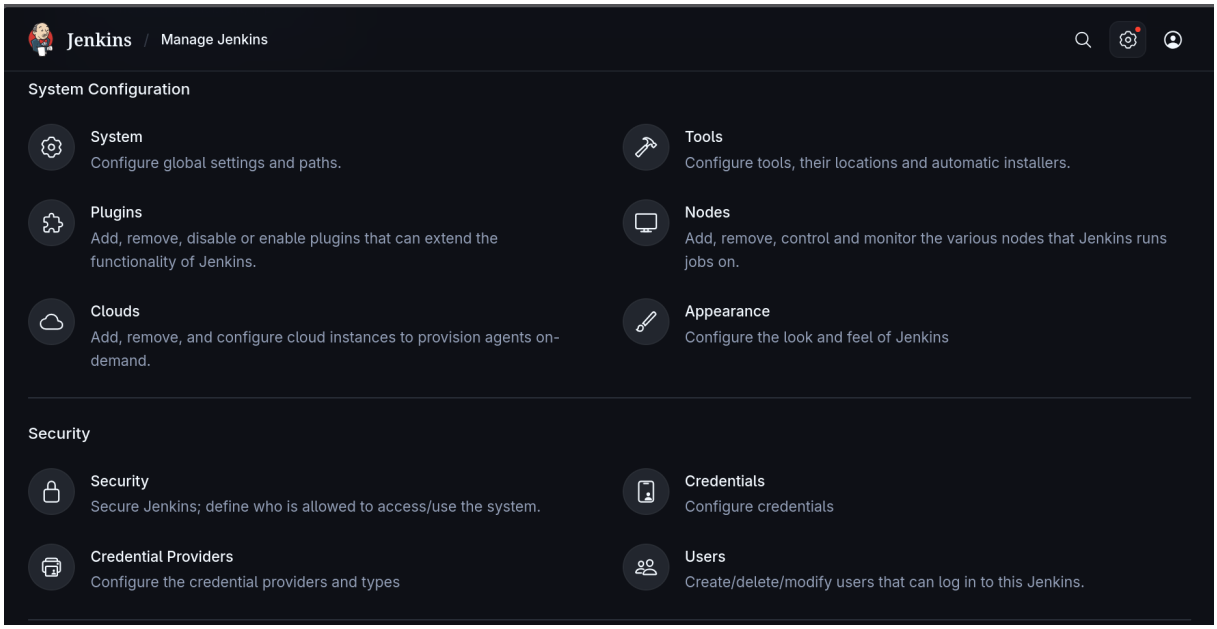


Figure 2 — Jenkins "Manage Jenkins" page (system configuration, plugins, credentials).

Sprint 2 — SonarQube and Code Quality

Objective: automate static code analysis.

Work done

- Started SonarQube (Community Edition 9.9) as a Docker container on port 9000 and changed the admin password.
- Generated an authentication token for Jenkins (kept secret, stored as a Jenkins credential).
- Installed the SonarQube Scanner plugin and declared the server under *Manage Jenkins* → *System*.
- Created a webhook SonarQube → Jenkins so the Quality Gate result is returned to the pipeline.

Deliverable: Jenkins and SonarQube communicate; analysis results appear in the SonarQube dashboard.

Figures

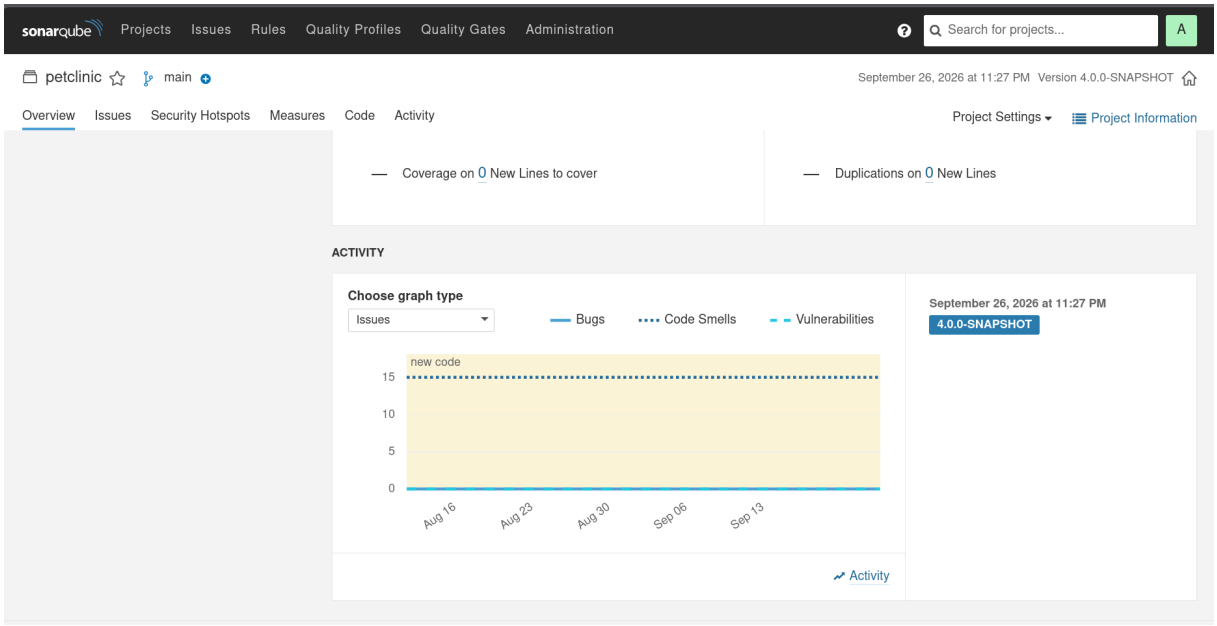


Figure 3 — SonarQube project "petclinic" — activity graph after the first analysis.

Sprint 3 — Base Jenkins Pipeline (Build + Quality Gate)

Objective: obtain the first working pipeline.

Work done

- Forked the Spring PetClinic repository on GitHub.
- Created a Jenkins *Pipeline* job that reads the *Jenkinsfile* from Git.
- Wrote the stages **Git Checkout**, **Build** (Maven), **SonarQube Analysis** and **Quality Gate**.
- Ran the pipeline and fixed the errors found (Groovy syntax error in the Jenkinsfile, Checkstyle violations caused by leftover report files).

Deliverable: Pipeline green up to the SonarQube analysis and Quality Gate.

Figures

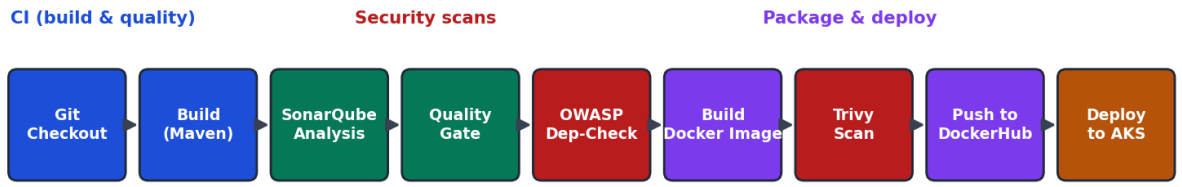


Figure 4 — Final pipeline design — the first four stages were delivered in this sprint.

Sprint 4 — Security: OWASP Dependency-Check and Trivy

Objective: scan for vulnerabilities before deployment.

Work done

- Installed the OWASP Dependency-Check plugin and added an NVD API key as a Jenkins credential.
- Added the stage **OWASP Dependency Check** that publishes an HTML/XML report and a trend graph.
- Added **Build Docker Image** (with a *.dockerignore* file so reports are not copied into the image).
- Added **Trivy Scan** on the image (timeout raised to 15 minutes) and **Push to DockerHub** using stored credentials.

Deliverable: Complete CI + security pipeline; scanned image published on DockerHub (build #30 fully green).

Figures

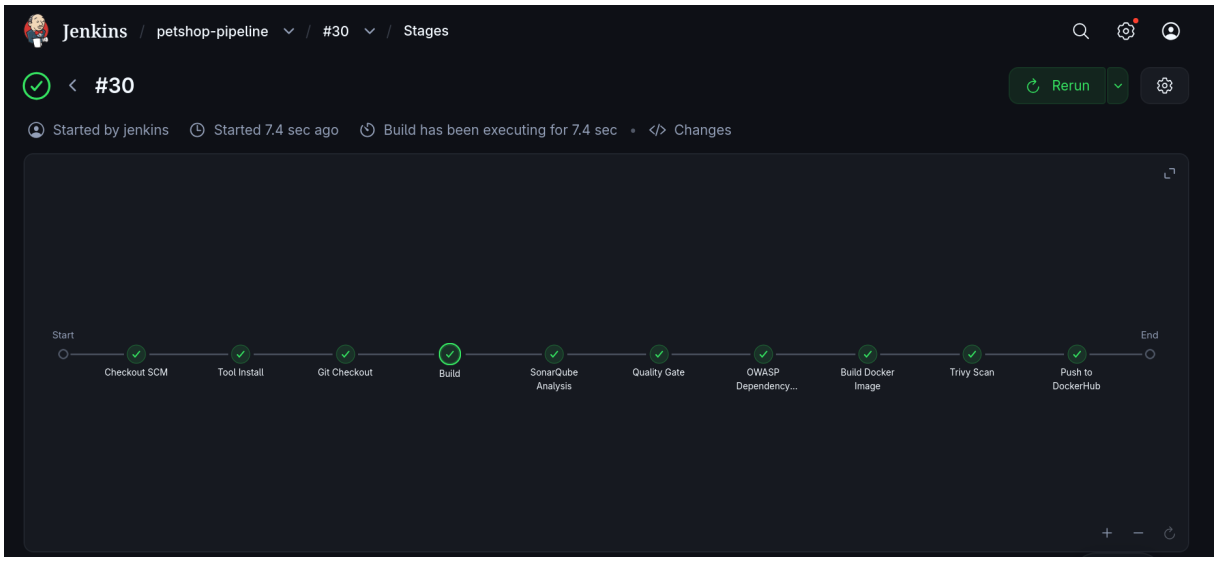


Figure 5 — Jenkins Stage View, build #30 — all stages successful.



Figure 6 — Job page: SonarQube Quality Gate "Passed" and Dependency-Check trend graph.

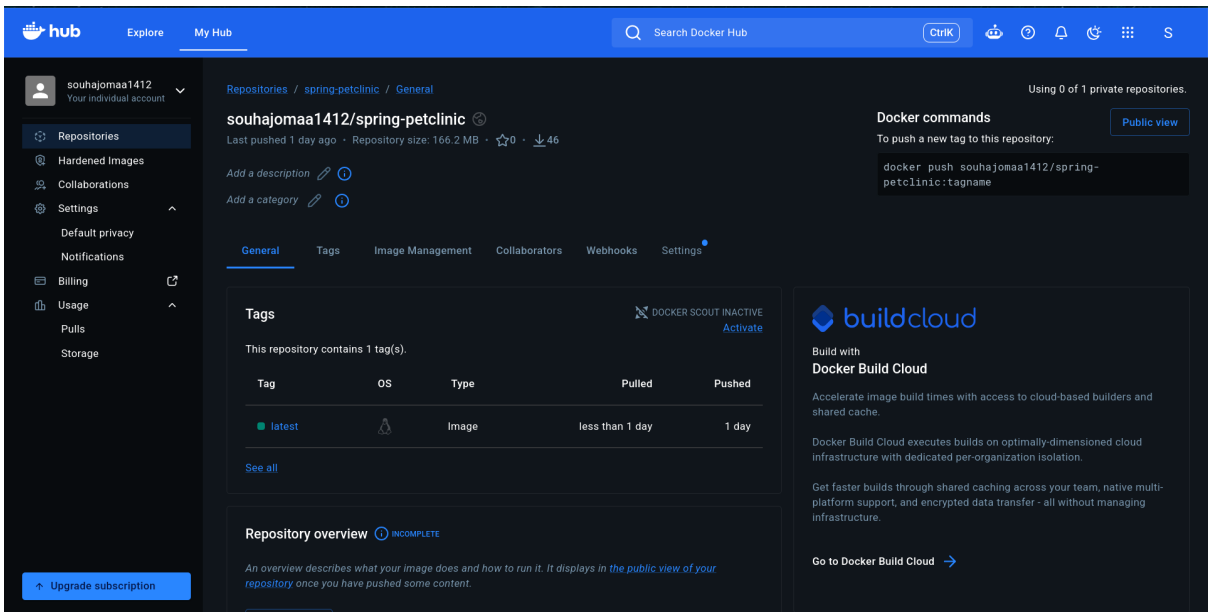


Figure 7 — DockerHub repository souhajomaa1412/spring-petclinic after the push.

Sprint 5 — Kubernetes Cluster (kubeadm)

Objective: build a functional Kubernetes cluster on the two remaining VMs.

Work done

- Prepared both nodes: swap disabled, kernel modules *overlay* and *br_netfilter*, sysctl network settings.
- Installed **containerd** (SystemdCgroup enabled) and **kubeadm / kubelet / kubectl** v1.30.
- Initialised the control-plane with `kubeadm init` on k8s-master and configured kubectl.
- Installed the **Calico** CNI plugin so pods can communicate.
- Joined k8s-worker with `kubeadm join`.

Deliverable: Two-node Kubernetes cluster operational (both nodes Ready).

Figures

```
vagrant@k8s-worker:~$ sudo kubeadm join 192.168.56.11:6443 --token <hidden> ...
[preflight] Running pre-flight checks
[kubelet-start] Starting the kubelet
[kubelet-check] The kubelet is healthy after 507.213563ms
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.
```

Figure 8 — Worker node joining the cluster (token hidden).

```
vagrant@k8s-master:~$ kubectl get nodes
NAME         STATUS   ROLES           AGE     VERSION
k8s-master   Ready    control-plane   13m     v1.30.14
k8s-worker   Ready    <none>          2m30s   v1.30.14
```

Figure 9 — kubectl get nodes — two nodes in Ready state.

```
vagrant@k8s-master:~$ kubectl get pods -n kube-system
NAME                                   READY   STATUS    RESTARTS   AGE
calico-kube-controllers-564985c589-tt45b   1/1     Running   0          5m33s
calico-node-kzk5p                      1/1     Running   0          5m33s
coredns-55cb58b774-94xml               1/1     Running   0          12m
coredns-55cb58b774-qtwl5               1/1     Running   0          12m
etcd-k8s-master                        1/1     Running   0          12m
kube-apiserver-k8s-master              1/1     Running   0          12m
kube-controller-manager-k8s-master     1/1     Running   0          12m
kube-proxy-vtq2z                       1/1     Running   0          12m
kube-scheduler-k8s-master              1/1     Running   0          12m
```

Figure 10 — System pods in the kube-system namespace (Calico, CoreDNS, etcd, API server...).

Sprint 6 — Ansible: Automated Deployment

Objective: let Ansible deploy the application on Kubernetes.

Work done

- Generated an SSH key on jenkins-sonar and authorised it on the master and the worker.
- Installed Ansible and created an *inventory.ini* with the two hosts; `ansible all -m ping` succeeded.
- Wrote the Kubernetes manifest (Deployment with 2 replicas + NodePort Service on port 30080).
- Wrote the playbook *kube.yaml* that copies the manifest to the master and runs `kubectl apply`.
- Solved a temporary *ImagePullBackOff* caused by slow DNS on the worker; the image was then pulled successfully.

Deliverable: Application deployed automatically on the cluster and reachable from the browser.

Figures

```
vagrant@jenkins-sonar:~$ ssh vagrant@192.168.56.12 "hostname"
The authenticity of host '192.168.56.12 (192.168.56.12)' can't be established.
ED25519 key fingerprint is SHA256:d6C73eArzetlHoOGzYS1La6Qvv7BEFUOMtUvA02nD78.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.12' (ED25519) to the list of known hosts.
k8s-worker
vagrant@jenkins-sonar:~$
```

Figure 11 — Passwordless SSH from jenkins-sonar to k8s-worker.

```
vagrant@jenkins-sonar:~/ansible/petshop$ ansible all -i inventory.ini -m ping
192.168.56.12 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
192.168.56.11 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
vagrant@jenkins-sonar:~/ansible/petshop$
```

Figure 12 — Ansible ping — SUCCESS on both Kubernetes nodes.

```
vagrant@k8s-master:~$ kubectl get pods -o wide
NAME                                 READY   STATUS    RESTARTS   AGE   IP               NODE         NOMINATED NODE   READINESS GATES
petshop-deployment-5f67f96bd6-4t2ls   1/1     Running   0          10m   192.168.254.130   k8s-worker   <none>           <none>
petshop-deployment-5f67f96bd6-cmm4z   1/1     Running   0          10m   192.168.254.129   k8s-worker   <none>           <none>
vagrant@k8s-master:~$ kubectl get endpoints petshop-service
NAME              ENDPOINTS                                   AGE
petshop-service   192.168.254.129:8080,192.168.254.130:8080   13m
vagrant@k8s-master:~$ kubectl get svc
NAME              TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
kubernetes        ClusterIP   10.96.0.1       <none>        443/TCP          24h
petshop-service   NodePort    10.111.145.11   <none>        8080:30080/TCP   13m
vagrant@k8s-master:~$
```

Figure 13 — Two petshop pods Running on k8s-worker, with their service endpoints.

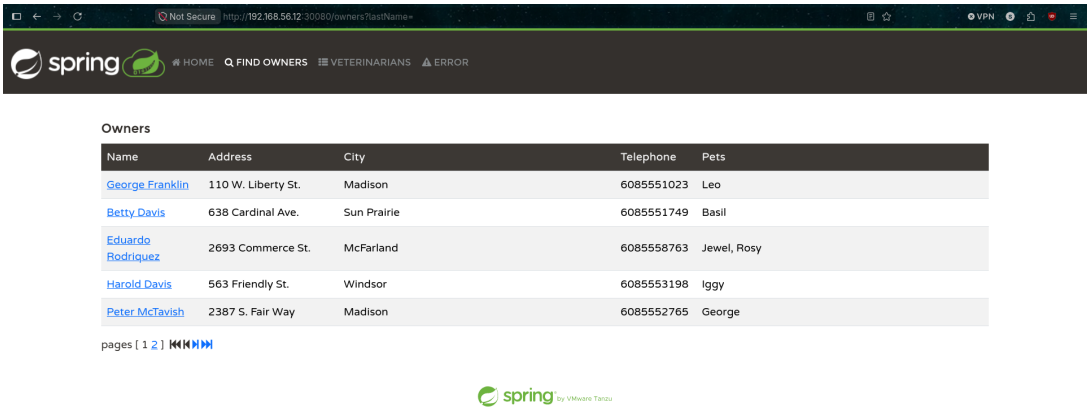


Figure 14 — Spring PetClinic running on the local Kubernetes cluster (NodePort 30080).

Sprint 7 — Verification and Monitoring

Objective: confirm the application works and add observability.

Work done

- Checked all resources with `kubectl get all`; accessed the application through the NodePort.
- Installed **Helm** and deployed the *kube-prometheus-stack* chart in the *monitoring* namespace (Prometheus, Alertmanager, Grafana, node-exporter, kube-state-metrics).
- Exposed Grafana as a NodePort service (port 31538) and logged in as administrator.
- Opened the pre-built dashboard *Kubernetes / Compute Resources / Cluster* showing live CPU and memory data.

Deliverable: End-to-end DevSecOps pipeline with the application visible and monitored on the local cluster.

Figures

```
vagrant@k8s-master:~$ kubectl get all -n monitoring
NAME                                                     READY   STATUS    RESTARTS      AGE
pod/alertmanager-monitoring-kube-prometheus-alertmanager-0   2/2     Running   2 (69s ago)   41m
pod/monitoring-grafana-56b964577b-w46lr                  3/3     Running   0             43m
pod/monitoring-kube-prometheus-operator-9b676f8d-bgnrw   1/1     Running   3 (71s ago)   43m
pod/monitoring-kube-state-metrics-6fc9876b6-8kpgz        1/1     Running   7 (69s ago)   43m
pod/monitoring-prometheus-node-exporter-49msp            1/1     Running   3 (70s ago)   43m
pod/monitoring-prometheus-node-exporter-t8825            1/1     Running   1 (22m ago)   43m
pod/prometheus-monitoring-kube-prometheus-prometheus-0   2/2     Running   3 (66s ago)   41m

NAME                                                TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)                      AGE
service/alertmanager-operated                       ClusterIP   None             <none>        9093/TCP,9094/TCP,9094/UDP   41m
service/monitoring-grafana                          NodePort    10.103.152.155   <none>        80:31538/TCP                 43m
service/monitoring-kube-prometheus-alertmanager     ClusterIP   10.96.90.113     <none>        9093/TCP,8080/TCP            43m
service/monitoring-kube-prometheus-operator         ClusterIP   10.111.35.164    <none>        443/TCP                      43m
service/monitoring-kube-prometheus-prometheus       ClusterIP   10.106.209.17    <none>        9090/TCP,8080/TCP            43m
service/monitoring-kube-state-metrics               ClusterIP   10.111.112.42    <none>        8080/TCP                     43m
service/monitoring-prometheus-node-exporter         ClusterIP   10.98.5.148      <none>        9100/TCP                     43m
service/prometheus-operated                         ClusterIP   None             <none>        9090/TCP                     41m

NAME                                                DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR            AGE
daemonset.apps/monitoring-prometheus-node-exporter   2         2         2       2            2           kubernetes.io/os=linux   43m

NAME                                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/monitoring-grafana                  1/1     1            1           43m
deployment.apps/monitoring-kube-prometheus-operator   1/1     1            1           43m
deployment.apps/monitoring-kube-state-metrics       1/1     1            1           43m

NAME                                                           DESIRED   CURRENT   READY   AGE
replicaset.apps/monitoring-grafana-56b964577b                  1         1         1       43m
replicaset.apps/monitoring-kube-prometheus-operator-9b676f8d   1         1         1       43m
replicaset.apps/monitoring-kube-state-metrics-6fc9876b6        1         1         1       43m

NAME                                                               READY   AGE
statefulset.apps/alertmanager-monitoring-kube-prometheus-alertmanager   1/1     41m
statefulset.apps/prometheus-monitoring-kube-prometheus-prometheus      1/1     41m
vagrant@k8s-master:~$
```

Figure 15 — Pods, services and deployments of the monitoring stack.

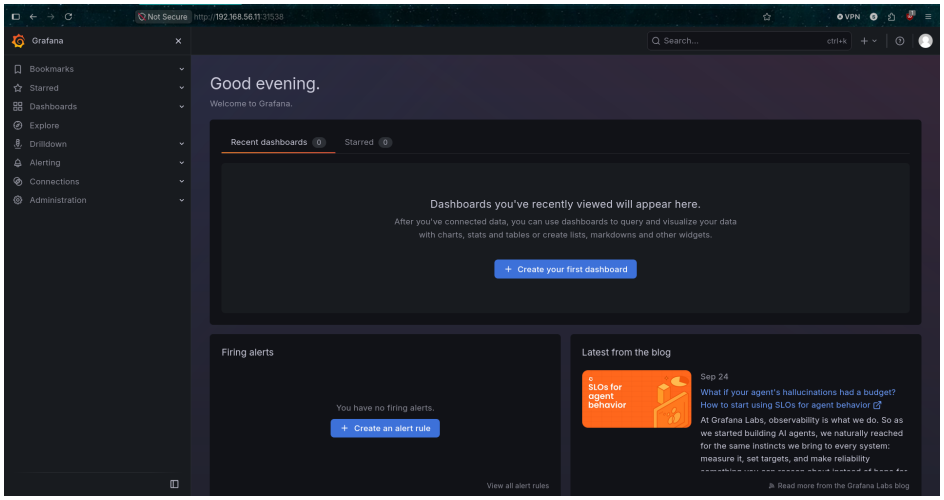


Figure 16 — Grafana welcome page after the first login.

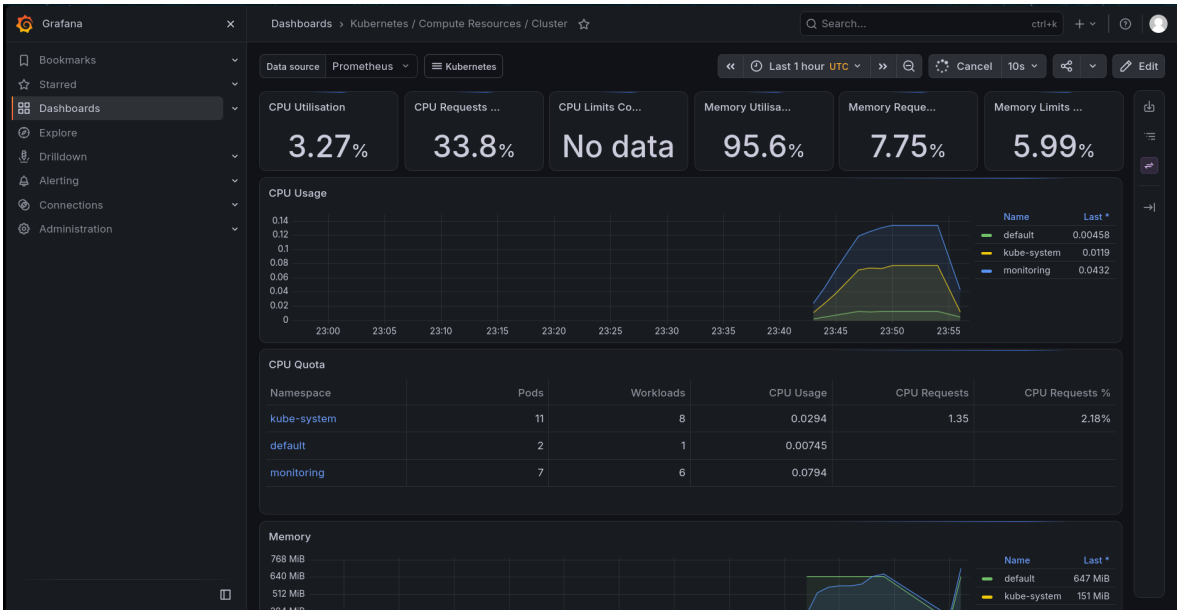


Figure 17 — Grafana dashboard "Kubernetes / Compute Resources / Cluster".

Sprint 8 — Deployment on Azure (AKS)

Objective: run the application on a managed cloud Kubernetes cluster and let Jenkins deploy to it automatically.

Work done

- Signed in with the Azure CLI (*Azure for Students* subscription), registered the required resource providers and created the resource group *rg-souha* in France Central.
- Checked the vCPU quota and VM-size availability: *Standard_B2s* was not offered in the region, so the cluster uses *Standard_B2ls_v2* nodes in availability zone 1.
- Created the AKS cluster *aks-devsecops* (Free tier, 2 nodes, 32 GB OS disks) with `az aks create` and connected kubectl with `az aks get-credentials`.
- Wrote *k8s/deployment.yaml*: Deployment *spring-devsecops* (2 replicas, CPU/memory requests and limits) and a **LoadBalancer** Service on port 80; committed it to the GitHub repository.
- Deployed Prometheus and Grafana with Helm (kube-prometheus-stack; Alertmanager and control-plane scrapers disabled because AKS manages the control plane). Grafana is reached through a port-forward tunnel and is not exposed to the Internet.
- Added the cluster kubeconfig to Jenkins as a *Secret file* credential and a new pipeline stage **Deploy to AKS** (`kubectl apply`, `rollout restart`, `rollout status`). Build #32 was fully green.
- Controlled the cost of the student credit by stopping the cluster when idle (`az aks stop`).

Deliverable: Application publicly reachable on AKS, deployed automatically by Jenkins after every build, and monitored with Prometheus/Grafana.

Figures

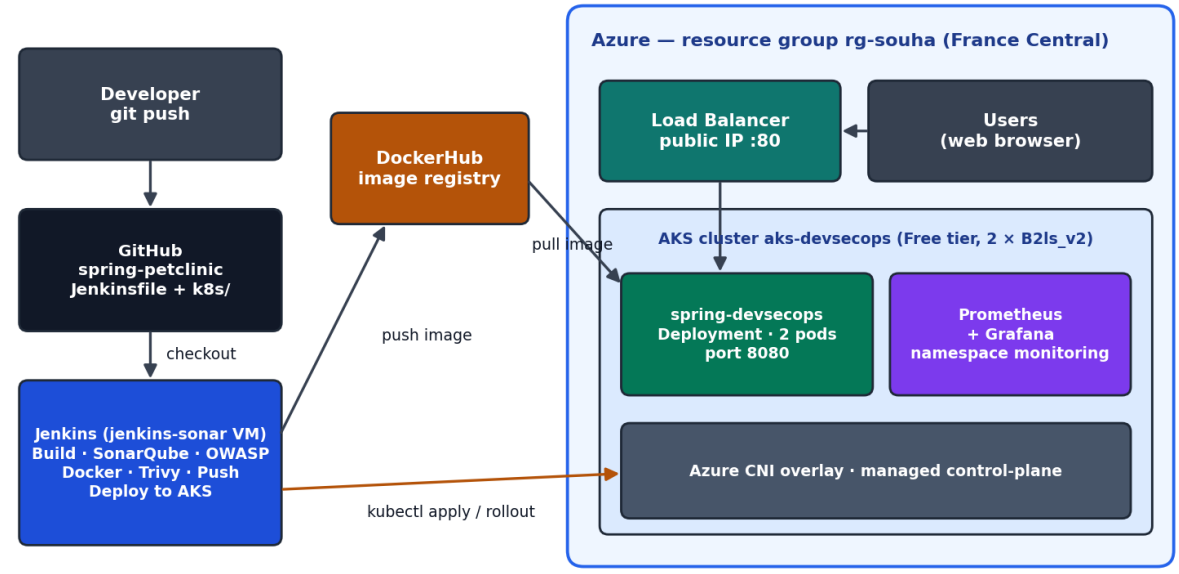


Figure 18 — Target architecture on Azure: Jenkins builds and scans, DockerHub stores the image, AKS runs it.

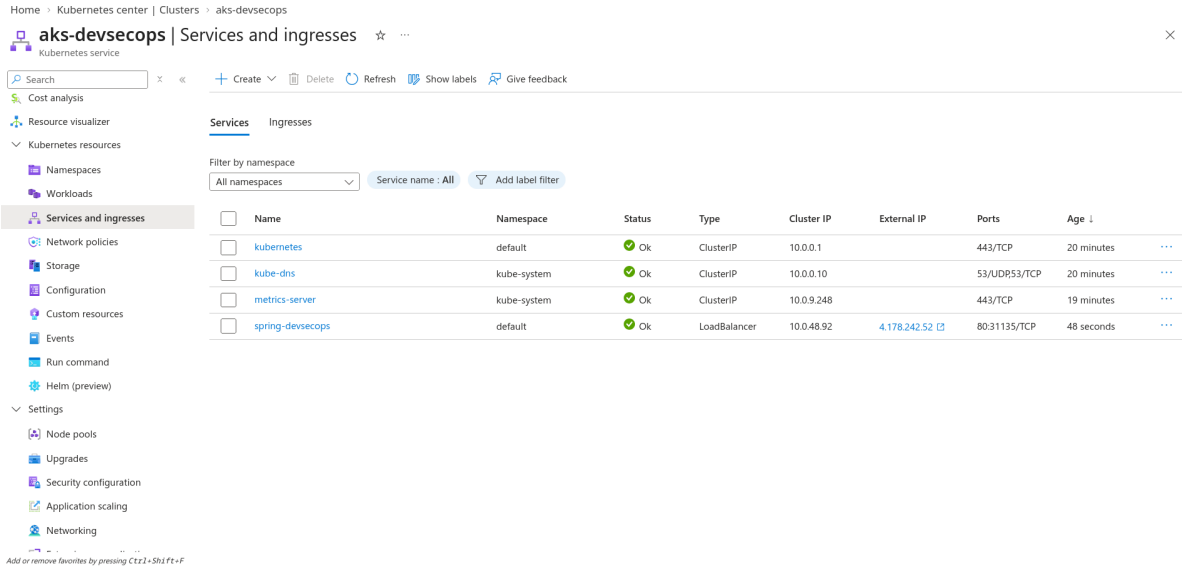


Figure 19 — Azure portal — Services and ingresses: LoadBalancer service spring-devsecops with its public IP.

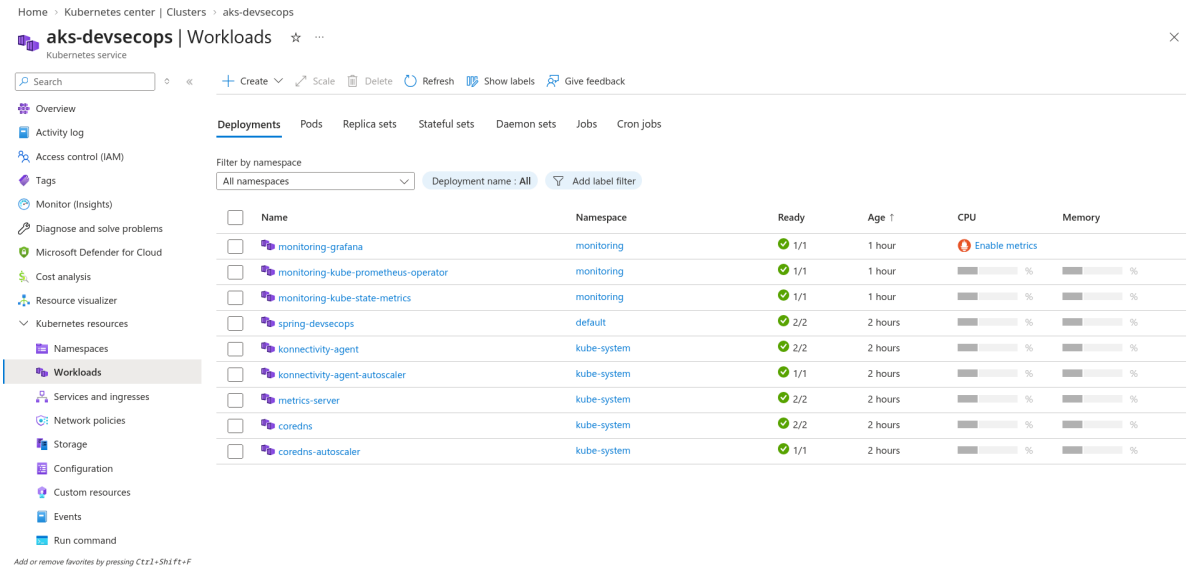


Figure 20 — Azure portal — Workloads: spring-devsecops (2/2) and the monitoring stack are healthy.

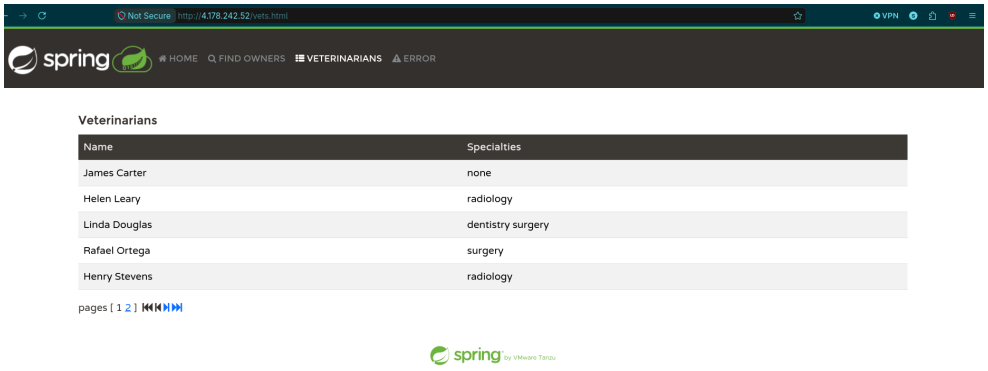


Figure 21 — Spring PetClinic served from AKS through the public load balancer.

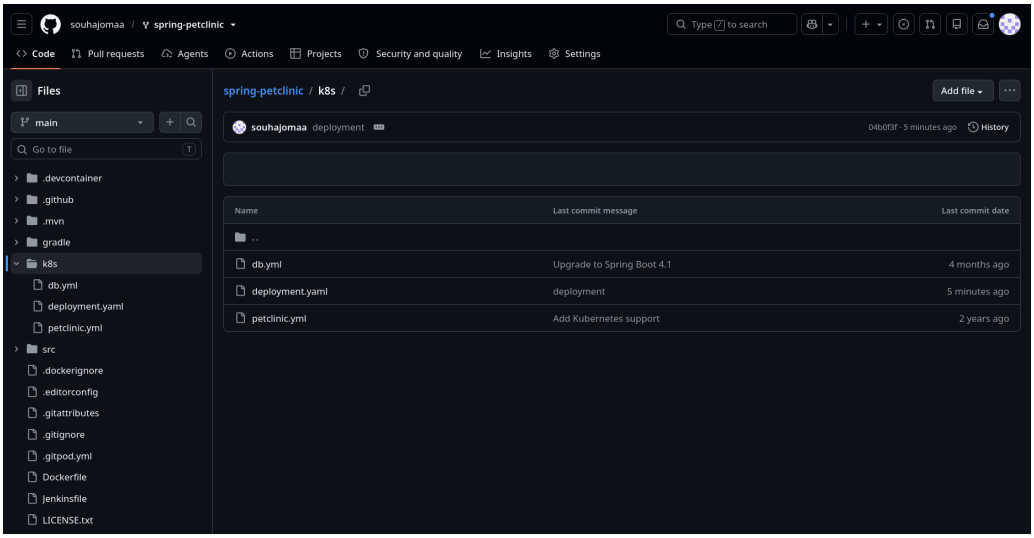


Figure 22 — GitHub repository: the k8s/ folder with deployment.yaml.

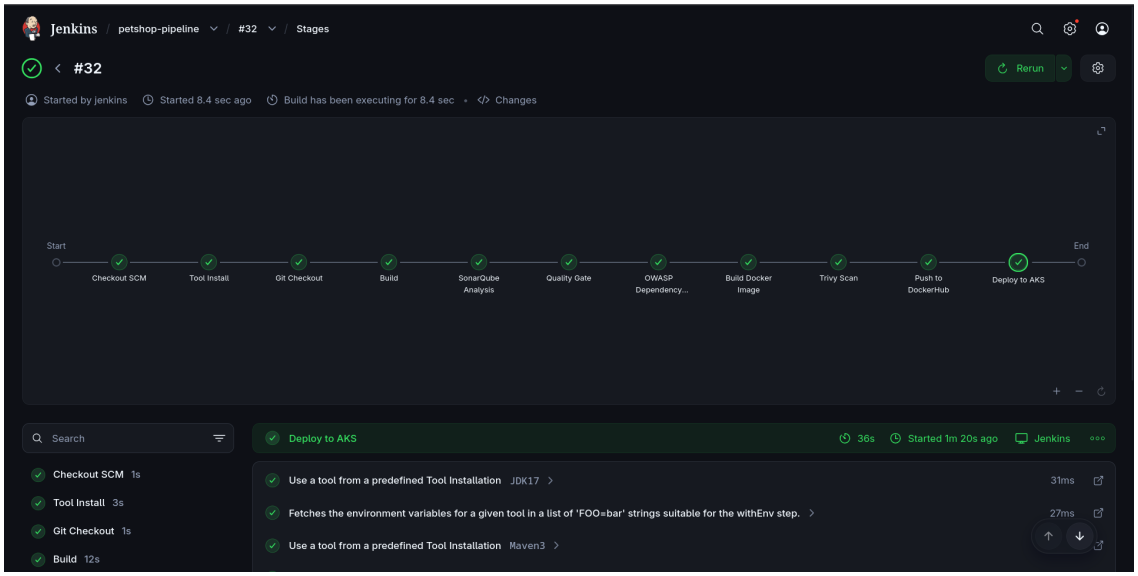


Figure 23 — Jenkins build #32 — all stages green, including Deploy to AKS.

```
souhajo@fedora:~/Documents/soujo$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
spring-devsecops-84bfb6bdf-7t2rv    1/1     Running   0          2m52s
spring-devsecops-84bfb6bdf-dghc9    1/1     Running   0          3m
souhajo@fedora:~/Documents/soujo$
```

Figure 24 — kubectl get pods after the pipeline run: freshly restarted pods (age of about 3 minutes).

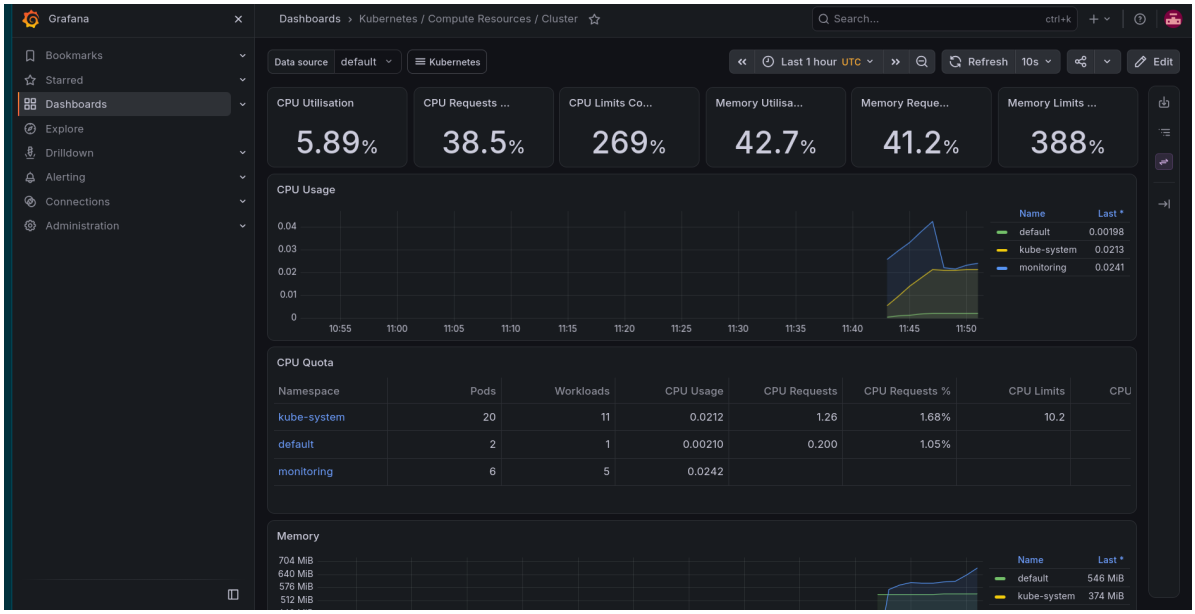


Figure 25 — Grafana on AKS — Kubernetes / Compute Resources / Cluster.

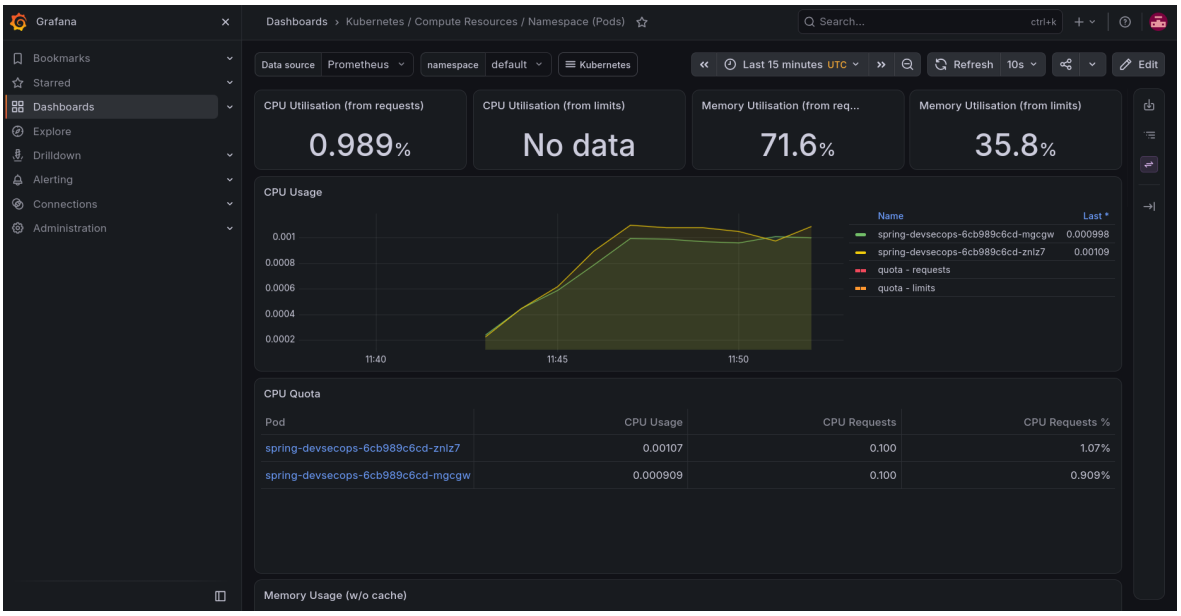


Figure 26 — Grafana — Namespace (Pods): CPU usage of the two spring-devsecops pods.

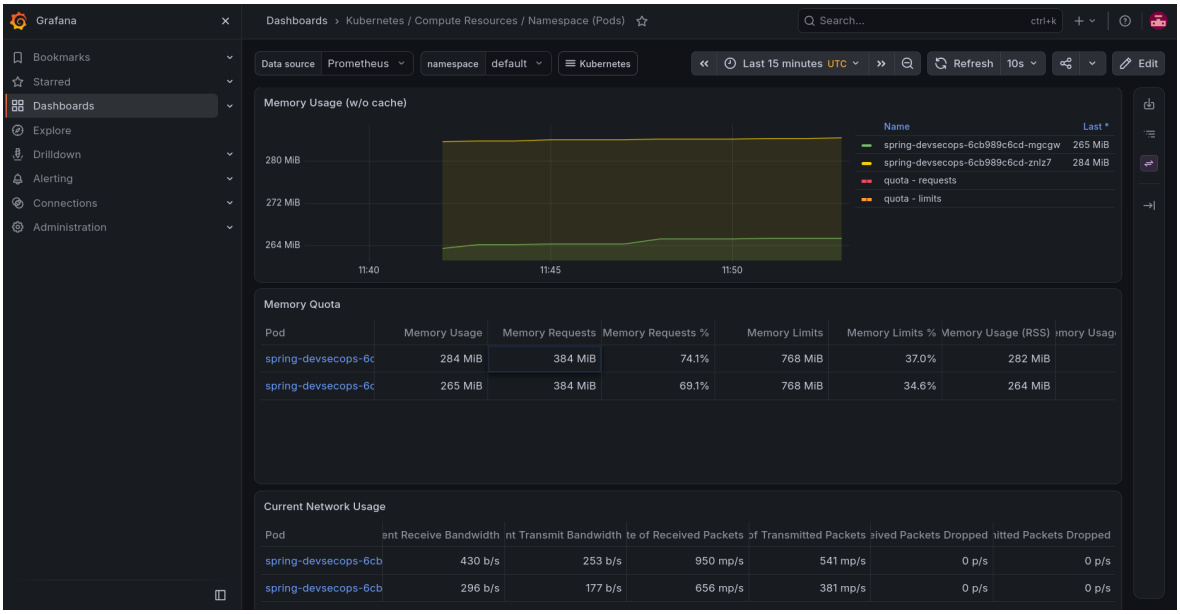


Figure 27 — Grafana — memory usage, quotas and network of the application pods.

Conclusion and Lessons Learned

The project delivers a working DevSecOps chain: source code → build → quality analysis → security scans → container image → automated deployment on Kubernetes (local, then Azure AKS) → monitoring. Main problems met and their solutions:

Problem	Cause	Solution
Pipeline failed to start	Extra closing brace in the Jenkinsfile	Corrected the Groovy syntax
Checkstyle violations (133)	Report files copied into the build context	Removed reports, added .dockerignore
Trivy scan timed out	Slow first download of the vulnerability DB	Added --timeout 15m
SSH key refused	Password login disabled on the VMs	Copied the public key manually
ImagePullBackOff	Slow / unstable DNS on the worker	Waited for retry; pull succeeded
dpkg lock on apt install	unattended-upgrades running at boot	Waited for the lock to be released
AKS VM size refused	Standard_B2s not offered in the region for the subscription	Used Standard_B2ls_v2 in zone 1
Helm install timeout on AKS	API server slow just after cluster creation	Retried the same command; it succeeded
Jenkins: tool "Maven3" not found	Tool entry renamed during the TP setup	Re-created the Maven3 tool in Manage Jenkins → Tools

Possible improvements

- Use immutable image tags (the Jenkins build number) instead of *latest*, so each deployment is traceable.
- Move images to Azure Container Registry and publish the application behind an Ingress with HTTPS.
- Add Grafana alerts and application-level metrics (Spring Boot Actuator).
- Add automated tests and code coverage (currently 0 % in SonarQube).
- Automate the Azure infrastructure with Terraform and schedule the cluster stop/start to save credit.